Zenith: learning executable observation models

Will Cook • 14 October 2026

Abstract. Zenith retains executable models of a source's responses, their supporting observations and counterexamples. A language model proposes experiments and writes restricted Python; a host executes both. Research returns motivate small tests on actual SQLite behavior. An initial comparison scored 244/288, but unchanged early code tied the final learner: continued revision added no measured benefit. Fresh reconstruction subsequently repaired selected failures. A four-candidate pool combining DeepSeek and Qwen programs, selected using observed development cases, scored 279/288 on separate confirmation sequences, versus 191/288 and 194/288 for single-model pools. A stronger later control selected from the larger historical archive and reached 286/288 on another panel after two native probes per source, with no new model calls. This favors preserving evidence and executable alternatives before purchasing new code. These conditional results do not establish graph superiority or recursive self-improvement.

1. Retain a model that observations can challenge. Let $D_s = \{(a_i, o_i)\}$ be host-recorded experiments on source s. An executable program P_s uses the evidence to predict a requested quantity q after a new action sequence a:

$$P_s(a, q, D_s) \longmapsto (v, u), \qquad u = 1 \implies v = \text{null}.$$

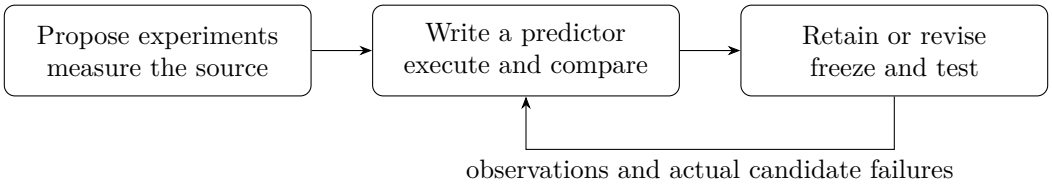
The retained object is code, source binding, measured receipts and counterexamples. The model's scope statement remains a hypothesis. A hash binds bytes; it does not show that the source's meaning is unchanged. Identifiability is specific to the question: uncertainty about one counter need not prevent a supported conclusion about committed state. After a model is refuted, however, partial reuse needs a justified component contract; surviving marginal fits alone do not establish a valid joint model.

A source whose numbers have different meanings. Each experiment starts with an empty SQLite table and two writing connections. The public actions are insertion, update, deletion and transaction control. The hidden source configuration varies conflict handling, an insertion trigger and counter selection. A separate connection measures committed count and value sum. The emitted counter can count direct changes, connection-total changes, or the totals across both writers. These are actual SQLite executions, not synthetic labels produced by a model. Native counters differ in whether they include trigger effects or another connection's writes [4].

For example, two insertions of the same key can preserve, replace or update a value. Rolling back can remove visible effects while a change counter retains work already performed. A no-op update can increment a counter without changing the stored value. Past measurements must distinguish these possibilities before a later compound sequence can be answered reliably. The model receives action documentation, the allowed source family and calibration receipts. Actual settings, candidate programs and the source implementation stay private.

The study supplies three useful quantities: channel, committed count and sum. It does not claim that the model invented those goals. The private evaluator executes every compatible configuration in a frozen 18-member family. A query is supported if their values agree. This identification condition is relative to that family, not a guarantee against every source consistent with finite data.

2. One investigation loop. The model proposes discriminating experiments. The host records their actual outcomes and an independent challenge. It then executes a proposed predictor on every observed development case. The next request receives the selected program and the actual last challenger, including rejected code and measured execution failures. Search can also restart from the same measurements without the failed code. Evidence persists across replacement; a candidate is retained only under the declared development rule. Lab, Oracle and Evolve name functions of this loop, not mandatory separate model agents.



Initial and repair trajectories retain the latest executable candidate whose development score is at least the incumbent's. Ties permit revision without claiming improvement; regressions remain recorded. A lookup can fit development, so further-case evaluation is essential. The initial six-source comparison matches observation history, tools and six proposals per source across incremental, archive and fresh construction. An unchanged early program with final history tests whether a stable interpreter already explains the benefit.

Repair and candidate search. Three failed programs are selected retrospectively. Generic repair, counterexample emphasis and cold reconstruction receive the same observations and two proposals, using DeepSeek and Qwen separately. A follow-on compares four-slot pools: two original proposals from each model, versus four from either. The latter add two continuations, so search histories differ. Development accuracy selects programs; source hashes break ties. New action structures exclude earlier calibration and evaluation structures. A uniform primary pool is committed before separate confirmation of all three pools.

Feedback without new code. A further test freezes eight attempt slots per source and acquires two probes in fixed order or by candidate disagreement. Within each history, compare the unchanged initial selection with development-based reselection. A stronger reference selects from all 18 same-source historical proposals using the same fixed-policy receipts; its larger search budget is explicit. The primary measure is correctly resolved supported answers on a fixed query denominator. Wrong claims, abstentions and execution errors are separate. This authored policy tests information and selection, not learning the investigation policy itself.

Learned composition. A model writes transition/readout functions, then graph, flat-library and fresh-library conditions receive the same functions, observations and three assembly proposals. Native nested savepoints test new behavior. The flat condition may inline or rewrite any code. Actual invocation, state consumption and neutral-child ablations distinguish useful dependence from merely naming a child.

All construction is frozen before protected scoring; failed outputs remain unchanged. Sources are configured members of one disclosed software family, not independent domains. Different screens are not pooled into one accuracy.

3. Evidence and limits. Earlier supplied prototypes passed 337 tests. Subsequent returns motivate state, interface and experiment acquisition. The latest fifteen were inspected, not rerun; their reported results remain separate from our measurements below.

Compilation, repair and complementary search. The initial 126-call DeepSeek comparison cost $0.1295. Incremental, archive and fresh construction scored 244, 187 and 230 of 288. Unchanged early code also scored 244; source-level comparison intervals span zero. Continued revision added no measured benefit. Two 18-call repair screens cost $0.0610. A selected DeepSeek restart confirmed at 188/288 versus 23/288 for unchanged failed seeds. A later four-slot comparison confirmed mixed DeepSeek/Qwen candidates at 279/288, versus 191/288 and 194/288 for single-model pools. Its 12 additional calls cost $0.0240. All mixed winners were retained original proposals. Counts match, but generation histories and costs differ; these are three selected failures.

Reuse the full archive before generating more. A later assay makes no model calls and measures 12 native probes across three sources. Initial and reselected programs share each policy's exact receipts. All choices freeze before a new final panel:

Inventory, probes, selection	Supported correct / 288	Exact / 288
Eight, fixed, initial	165	165
Eight, fixed, reselect	180	180
Eight, disagreement, initial	234	255
Eight, disagreement, reselect	249	270
Historical eighteen, fixed, initial	265	265
Historical eighteen, fixed, reselect	286	286

All conditions have zero execution errors and unsupported commitments. Disagreement leaves 21 queries unidentified; its exact score credits their correct abstentions, while the primary numerical score does not. Its entire 69-answer advantage comes from avoiding a fixed probe that exposes shared simulator misspecification. After that contradiction, the smaller inventory abstains on all 96 queries for that source. This is not better identification. The historical archive handles the stronger evidence, but has a larger search budget. This assay has no separate confirmation; its panel differs from the candidate-pool comparison.

Interpretation. Retain evidence, executable alternatives and independent challenges. Shared agreement can conceal a bug; search may preserve regressing intermediates. Five composition versions stopped before protected scoring. The next hypothesis is learned decomposition: contextual nodes revise missing state, interfaces or experiments. Compare discovery and later reuse against a competent agent allowed to refactor ordinary code. Equivalent compiled behaviour does not erase learned structure; a graph advantage requires lower effort or greater capability. This reflective mechanism remains untested. A feedback loop alone is not an improved learning method.

PAL [1], Voyager [2] and DreamCoder [3] motivate execution and reusable programs. Our comparison is limited to supplied SQLite goals and configurations. Panels are not pooled; archived replay preserves individual answers and failed attempts.

References

[1] L. Gao et al. *PAL: Program-aided Language Models*. 2022. arXiv:2211.10435.

[2] G. Wang et al. *Voyager: An Open-Ended Embodied Agent with Large Language Models*. 2023. arXiv:2305.16291.

[3] K. Ellis et al. *DreamCoder: Growing generalizable, interpretable knowledge with wake-sleep Bayesian program learning*. 2020. arXiv:2006.08381.

[4] SQLite. *Count the Number of Rows Modified; Total Number of Rows Modified*. changes; total_changes. Accessed 13 October 2026.